

reliomq Delivery Modes

User Guide and Deep Technical Explanation

Applies to reliomq 0.6.2

This document explains FastMode, GroupMode, and DurableMode, including message persistence, write/fsync behavior, DeliveryAck handling, checkpointing, power-loss behavior, replay risk, and storage-wear trade-offs.

1. Mode Overview

reliomq provides three delivery modes so an application can choose the balance between throughput, storage wear, and crash durability. The easiest way to understand the modes is to ask two questions: (1) when does the message become durable, and (2) when does the acknowledgement state become durable?

Important: a successful `write()` does not necessarily mean the bytes are physically durable. On Linux, ordinary writes typically enter the page cache first. `fsync()` is the explicit durability boundary used to force pending file data toward stable storage and wait for completion.

1.1 Quick comparison

Mode	Healthy-path payload	Data fsync	ACK persistence	Power-loss behavior	Best fit
Fast	RAM first; disk only when spilled/recovered	No per-message fsync on healthy path	RAM-only messages need no disk checkpoint; disk-backed completion is persisted	Newest RAM-only pending messages can be lost on sudden power loss	High-rate telemetry, SD-card-sensitive systems
Group	Append each message to Outbox immediately	Batched by count / bytes / interval	ACK cursor advances in RAM, then checkpointed in batches	Unsynced payload may be lost; recently ACKed messages may replay	Balanced industrial telemetry
Durable	Append message to Outbox	Per-message durable boundary	ACK completion checkpoint persisted individually	Strongest protection against accepted-message loss; minimal ACK replay window	Critical events / alarms / transactions

1.2 FastMode - speed and low storage wear

FastMode is RAM-first. During healthy operation, a message enters a bounded in-memory queue, is sent, and is removed after an authoritative `DeliveryAck`. If the message is ACKed before it needs to spill, the persistent Outbox is not used for that message.

FAST MODE - healthy path

```
publish()
|
v
RAM queue
|
v
send through MQTT
|
v
DeliveryAck
|
v
remove from RAM
```

Normal healthy path:

- no payload append to Outbox
- no payload fsync
- no ACK checkpoint for that RAM-only message

FastMode spills pending RAM messages to the durable Outbox when its spill policy requires it. The default policy includes bounded RAM limits, a high-watermark, maximum RAM age, disconnect grace, and batch spill limits. In 0.6.2, `high_watermark`, `max_ram_age`, and `disconnect_grace` accept `None` to disable only that trigger; hard RAM and batch limits remain required. A spill batch is written with a durable batch boundary.

1.3 GroupMode - balanced mode

GroupMode appends each accepted message to the persistent Outbox immediately, but it does not fsync every append. Instead, it groups data synchronization and ACK checkpoint synchronization separately.

GROUP MODE

Payload path:
msg1 -> append
msg2 -> append
msg3 -> append
|
+----> one data fsync when threshold/time is reached

ACK path:
ACK1 -> RAM cursor
ACK2 -> RAM cursor
ACK3 -> RAM cursor
|
+----> write checkpoint.json
fsync checkpoint

Default data synchronization triggers are 20 unsynced messages, 0.25 seconds, or 64 KiB of unsynced framed data. Default ACK checkpoint triggers are 50 ACK advances or 1.0 second; a completed closed segment can also force a checkpoint.

1.4 DurableMode - strongest persistence

DurableMode prioritizes persistence over throughput. Each persistent message crosses a durable fsync boundary, and ACK progress is also persisted individually before the completed message is forgotten from the persistent queue.

DURABLE MODE

message
|
v
append to Outbox
|
v
fsync data
|
v
send
|
v
DeliveryAck
|
v
write checkpoint

|
v
fsync checkpoint
|
v
completed

1.5 Recommended mode selection

Requirement	Recommended mode
Highest throughput and lowest SD-card activity; newest in-RAM data may be expendable	FastMode
Balanced reliability, throughput, and SD-card wear	GroupMode
An accepted message should survive immediate crash/power loss as strongly as practical	DurableMode

2. Deep Technical Detail

2.1 Storage architecture

The current Outbox is a segmented persistent FIFO. Payload records are stored in numbered segment files. The logical head of the queue is represented by a cursor containing a segment ID and a byte offset. Acknowledging a message advances this cursor; reliomq does not rewrite all remaining payload records and does not maintain an ever-growing per-message ACK journal.

Example Outbox directory

```
outbox/  
|-- format.json  
|-- checkpoint.json  
|-- segment-00000000000000000001.dat  
|-- segment-00000000000000000002.dat  
`-- ...
```

checkpoint.json concept:

```
{  
  "version": 1,  
  "generation": 37,  
  "segment_id": 2,  
  "byte_offset": 48291  
}
```

Meaning:

everything before the cursor has been logically completed;
records at/after the cursor remain pending.

2.2 What write, append, flush, and fsync mean

Operation	Meaning	Durability implication
append/write	Adds bytes to the file; on Linux this commonly enters the kernel page cache first.	Fast return; does not by itself guarantee the bytes are physically durable.
userspace flush	Moves buffered runtime/library data into the operating-system write path.	Still not the same durability guarantee as fsync.
fsync	Requests pending file data/metadata needed for the file to be pushed to stable storage and waits for completion.	This is the explicit durability boundary used by the library.
checkpoint	Persists the current FIFO head cursor, after synchronizing data needed through that cursor.	Makes ACK/completion progress durable and enables safe cleanup of consumed segments.

2.3 ACK model: DeliveryAck versus MQTT transport ACK

The completion discussed in this document is the authoritative reliomq DeliveryAck used by the application-level delivery protocol. It should not be confused with a transport-level MQTT PUBACK. A DeliveryAck tells the sender that the application-level delivery has been accepted and the oldest matching Outbox record may advance toward completion.

```
publish  
|  
v  
pending message  
|
```

```

v
MQTT transport
|
v
destination / relay
|
v
DeliveryAck
|
v
complete oldest matching record
|
v
advance Outbox cursor

```

2.4 checkpoint.json and ACK persistence

ACK progress is represented by the queue cursor, not by appending one ACK line for every message. The checkpoint file uses the fields `version`, `generation`, `segment_id`, and `byte_offset`. The generation increments each time a new checkpoint is successfully installed.

Before ACKs

```

segment:
[msg1][msg2][msg3][msg4]
^
checkpoint

```

After ACK of msg1:

```

[msg1][msg2][msg3][msg4]
^
new logical head

```

After ACK of msg2:

```

[msg1][msg2][msg3][msg4]
^
new logical head

```

For a durable checkpoint, the cursor state must first exist in memory, then be serialized/written to the checkpoint file and fsynced. fsync alone cannot persist a new cursor value that has never been written to a file.

2.5 FastMode in detail

2.5.1 Healthy path

FastMode publication starts in the Sender-owned bounded RAM queue. The persistence policy intentionally does not append the normal FastMode message to the Outbox first.

```

t0 publish(msg1)
   RAM: [msg1]
   Disk: unchanged

t1 send(msg1)

t2 DeliveryAck(msg1)
   RAM: []
   Disk: unchanged

```

2.5.2 Spill path

When the RAM queue must spill, FastMode uses a durable batch append. The batch is appended to the Outbox with sync enabled, creating one durable batch boundary rather than an fsync for every individual message in the spill batch.

RAM before spill:

[msg10][msg11][msg12][msg13]

spill batch

|

v

segment file:

[msg10][msg11][msg12][msg13]

|

v

durable batch sync

After recovery:

disk-backed messages are resent.

When DeliveryAck arrives, the Outbox cursor is persisted before completion returns.

2.5.3 Power-loss consequence

If power is lost while a message is still only in RAM, that message may be lost. Once a FastMode message has been spilled and durably synchronized, it follows disk-backed recovery behavior. This is the central trade-off that allows FastMode to minimize healthy-path storage writes.

2.6 GroupMode in detail

2.6.1 Payload append behavior

For GroupMode, each accepted message is appended with sync disabled. The policy tracks the number of unsynced messages and framed bytes, and, when enabled, starts a synchronization timer. A data sync occurs when any configured threshold is reached.

Defaults

sync_messages = 20

sync_interval = 0.25 seconds

sync_bytes = 64 KiB

Example at 5 messages/second:

t=0.00 append msg1 (dirty / unsynced)

t=0.20 append msg2 (dirty / unsynced)

t=0.25 timer can trigger data sync

-> Outbox.sync()

-> fsync pending segment data

The exact trigger depends on count, bytes, and time; whichever condition is reached first wins. In 0.6.2, any individual trigger accepts None, but at least one data-sync trigger and one ACK-checkpoint trigger must remain enabled.

2.6.2 ACK behavior

On `DeliveryAck`, `GroupMode` immediately removes the oldest matching message from the process-local pending queue with sync disabled. That advances the volatile cursor in RAM. The ACK count increases, and a separate ACK checkpoint timer/threshold determines when the cursor becomes durable.

Disk checkpoint says:

[msg1][msg2][msg3][msg4][msg5]
^

DeliveryAck msg1

DeliveryAck msg2

DeliveryAck msg3

RAM/volatile cursor:

[msg1][msg2][msg3][msg4][msg5]
^

checkpoint.json on disk can still point here:

^

Checkpoint trigger fires

|

v

Outbox.checkpoint()

1. syncs required segment data
2. writes new cursor checkpoint
3. fsyncs durable checkpoint state
4. can reclaim fully consumed closed segments

Now disk checkpoint points here:

^

Default ACK checkpoint triggers are 50 ACK advances or 1.0 second. Completion of a closed segment can also force a checkpoint so that safely consumed segments can be reclaimed.

2.6.3 Why replay can occur after power loss

Suppose msg1-msg3 were delivered and ACKed, but the volatile cursor had not yet been checkpointed. If power fails, RAM state disappears. On restart, `reliomq` reads the older durable checkpoint and can therefore replay msg1-msg3. This is intentional at-least-once behavior: replay is preferred over incorrectly forgetting a message whose completion was not durably recorded.

2.7 DurableMode in detail

2.7.1 Payload durability

`DurableMode` appends a persistent message using the Outbox's synchronized append path. The append returns only after the durable data boundary succeeds.

publish(msg1)

|

v

append msg1 to segment

|

v

fsync segment data

```

|
v
publish call can continue with msg1 durably recorded

```

2.7.2 ACK durability

After an authoritative DeliveryAck, DurableMode removes the oldest matching Outbox record using the synchronized completion path. The Outbox synchronizes required data, persists the next cursor, and only then removes the record from the live pending index.

```

DeliveryAck(msg1)
|
v
calculate next cursor
|
v
sync segment data if required
|
v
write new checkpoint.json
|
v
fsync checkpoint / atomic installation
|
v
remove msg1 from live pending queue
|
v
safe cleanup of completed closed segments

```

2.8 What the files may look like

The three modes do not have three different ACK file formats. Disk-backed messages use the same segmented Outbox and the same checkpoint format. The difference is frequency and timing.

Mode	Segment files	checkpoint.json	Typical update pattern
Fast	May remain untouched during healthy RAM-only delivery; used after spill/recovery	Only relevant to disk-backed pending records	No per-message healthy-path checkpoint
Group	Append every message; fsync in batches	Same cursor schema	Many ACKs can be represented by one checkpoint write/fsync
Durable	Append and synchronize each accepted persistent message	Same cursor schema	Completion cursor is persisted for each disk-backed ACK

2.9 Crash and restart scenarios

Scenario	FastMode	GroupMode	DurableMode
Power loss before message reaches disk	RAM-only message can be lost	Recent append may be lost if not yet synced	Accepted persistent message should already have crossed its sync boundary
Power loss after delivery but before ACK progress is durable	RAM-only ACKed message is already gone; disk-backed completion uses durable path	Message may replay after restart	Completion checkpoint is persisted individually, minimizing this window
Clean shutdown	Pending work can be durably spilled/handled by shutdown logic	Pending data sync and ACK checkpoint work are flushed	Persistent operations are already individually durable

2.10 Storage-wear interpretation

The number of Python append calls is not the same as the number of physical flash-program operations. Linux can cache and combine ordinary writes. fsync is the more important application-visible durability operation because it forces pending data through the storage stack and waits for completion. Therefore, GroupMode primarily reduces forced synchronization frequency, while FastMode can avoid persistent writes entirely on the healthy path.

Illustrative behavior for 5 messages/second

FastMode, healthy connection:

5 messages/s -> RAM -> ACK

persistent message fsync rate: normally 0

GroupMode:

5 appends/s

data sync timer default: 0.25 s

ACK checkpoint default: up to 1.0 s or 50 ACKs

(actual I/O depends on timing, batching, rotation, and storage)

DurableMode:

each persistent message crosses a data sync boundary

each disk-backed ACK crosses a completion checkpoint boundary

highest forced-sync pressure of the three modes

2.11 Why GroupMode has two independent durability windows

GroupMode separates payload durability from ACK-progress durability because they solve different failure problems. Data synchronization protects newly accepted payloads. ACK checkpointing protects knowledge that already-delivered payloads are finished. Keeping the thresholds independent allows throughput and replay behavior to be tuned separately.

DATA WINDOW

append msg -> page cache -> ... -> fsync

Risk before fsync: newest payload may be lost

ACK WINDOW

DeliveryAck -> RAM cursor -> ... -> checkpoint + fsync

Risk before checkpoint: delivered payload may be replayed

Result:

payload-loss window and duplicate-replay window are separate knobs.

2.12 Practical explanation for end users

For documentation or customer communication, the following wording is concise and technically accurate:

- **FastMode:** Keeps normal traffic in RAM for maximum speed and minimum storage wear. Messages are written to persistent storage only when spill/recovery behavior requires it. Best for high-rate telemetry where the newest unsaved RAM messages may be acceptable to lose during an abrupt power failure.
- **GroupMode:** Writes every accepted message to the Outbox but batches fsync operations and ACK checkpoints. It offers a balance between reliability and storage wear. A sudden power failure can lose the newest unsynced payloads or cause recently ACKed messages to be replayed.

- **DurableMode:** Persists each accepted message and each disk-backed completion state with an immediate durability boundary. It provides the strongest crash protection, with the highest synchronization cost and storage activity.

2.13 Glossary

Term	Meaning
Append	Add a new record to the end of the active segment file.
Dirty data	Modified file data held in memory/page cache that has not yet completed writeback to stable storage.
fsync	A system call used to force pending file data/metadata required for durability to storage and wait for completion.
Outbox	The persistent FIFO holding disk-backed messages that have not been durably completed.
Segment	A numbered data file containing framed message records.
Cursor	A (segment_id, byte_offset) position representing the logical head of the pending FIFO.
Checkpoint	The durable representation of the current cursor in checkpoint.json.
DeliveryAck	The application-level acknowledgement that authoritatively completes the oldest matching message.
Replay	Resending a message after restart because its completion was not durably checkpointed.
At-least-once	A delivery model that prioritizes not losing messages; duplicates can occur and consumers should be idempotent where needed.

Sources and implementation basis

This guide is based on the reliomq 0.6.2 main-branch implementation, including `reliomq/durability.py`, `reliomq/persistence.py`, and `reliomq/outbox.py`. Repository: <https://github.com/pitpib23/reliomq>

Implementation details can change in later versions. When behavior is operationally critical, verify the installed package version and corresponding source.